Feature Detection & Matching

Week 10 — Session 2: Describing, matching & rejecting outliers

Practical Image Processing

Lecture 10.2 ▪ 60–70 min

Recap of Session 1

- Good features are **distinctive**, **repeatable**, **local** — corners fit best.
- **Harris**: structure matrix M, score $R = \det M - k(\operatorname{tr} M)^2$; then NMS.
- A **descriptor** summarizes the patch as an invariant vector.

Today

Build the descriptor, **match** descriptors across images, and reject the wrong matches — then this week's deliverable.

Roadmap — Session 2

Building descriptors

Matching descriptors

Rejecting bad matches

Scale invariance & applications

In-class exercise

Wrap-up & deliverable

Building descriptors

From keypoint to descriptor vector

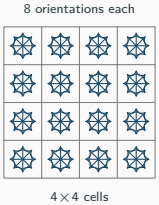
- Take an oriented, scaled patch around the keypoint (from Session 1's detector).
- Summarize it as a fixed-length vector so any two keypoints can be compared by distance.
- Two families: **float** histograms (SIFT) and **binary** strings (BRIEF/ORB).

Key idea

The descriptor is the "fingerprint" of a keypoint. Similar fingerprints $\Rightarrow$ the same physical point.

SIFT — gradient-orientation histograms

- Split the patch into a 4×4 grid of cells.
- In each cell, build an 8-bin histogram of gradient orientations (weighted by magnitude).
- Concatenate: $4 \times 4 \times 8 = \mathbf{128}$ numbers, then normalize.



Math

Orientations are measured *relative* to the keypoint's dominant angle $\Rightarrow$ rotation invariance.

SIFT — why 128 and why it is robust

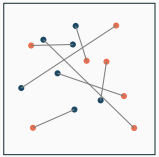
- Histograms tolerate small shifts: a gradient near a cell edge still lands in the right bin-ish.
- Normalizing the 128-vector to unit length removes contrast (multiplicative) changes.
- Clamping large values then re-normalizing curbs a few dominant gradients (non-linear lighting).

✓ Good practice

128 floats is a good trade: rich enough to be distinctive, small enough to match fast. This is the classic "gold standard" descriptor.

Binary descriptors — BRIEF

- Pick a fixed pattern of pixel *pairs* inside the patch.
- For each pair (a, b): write bit 1 if $I_a < I_b$, else 0.
- 256 pairs $\rightarrow$ a 256-bit string. Comparing = counting differing bits.



intensity-compare pairs

✕ Pitfall

Plain BRIEF is *not* rotation-invariant — rotate the patch and the pairs point elsewhere.

ORB = oriented FAST + rotated BRIEF

- **FAST** keypoints (Session 1) get an orientation from the patch intensity centroid.
- BRIEF's sampling pattern is *rotated* by that angle → rotation-aware ("rBRIEF").
- Result: a fast, free, rotation-invariant binary descriptor — the practical default.

Key idea

ORB gets most of SIFT's robustness at a fraction of the cost, with no patents. Great for real-time.

SIFT vs ORB (and friends)

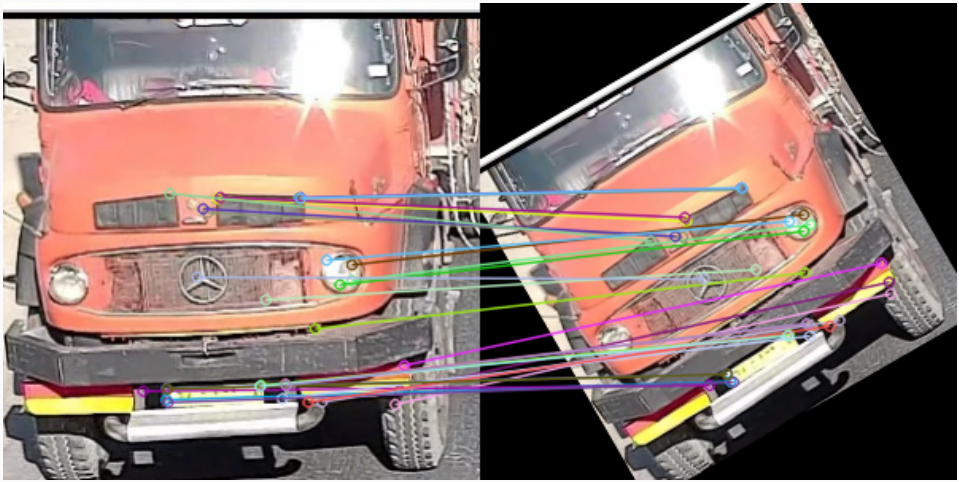
Descriptor	Type	Note
SIFT	128-D float	very robust; the gold standard
SURF	float	faster SIFT-like (patented)
ORB	binary (256-bit)	free, fast, great for real-time
BRIEF	binary	fast but not rotation-invariant alone

✔ Good practice

ORB is the practical default: free, fast, good enough. SIFT when you need maximum robustness.

Matching descriptors

Matching descriptors across images



Same car, rotated & scaled. ORB features still match — lines connect the same physical points. This is **invariance** working on real data.

How matching works

- For each descriptor in image A, find the **nearest** descriptor in image B (by distance).
- Float descriptors (SIFT): **Euclidean** distance. Binary (ORB): **Hamming** distance.
- **Brute-force** compares all pairs; **FLANN** approximates it fast for large sets.

Math

Hamming distance = number of differing bits — a single fast XOR + popcount for binary descriptors.

Hamming distance — worked example

Two 8-bit ORB descriptors:

$$a = 1011\,0010$$
$$b = 1001\,0110$$
$$a \oplus b = 0010\,0100$$

Popcount of $a \oplus b = 2 \Rightarrow$ Hamming distance $= \mathbf{2}$.

Fast in hardware

XOR + a single `popcount` instruction compares a whole 256-bit descriptor in a few cycles — why binary descriptors match so quickly.

Euclidean vs Hamming — pick by descriptor type

Descriptor	Distance	OpenCV norm
SIFT / SURF (float)	Euclidean (L_2)	`NORM_L2`
ORB / BRIEF (binary)	Hamming	`NORM_HAMMING`

✗ Pitfall

Using `NORM_L2` on a binary descriptor (or Hamming on floats) gives garbage matches — match the metric to the descriptor.

Brute-force, kNN, and FLANN

- **Brute-force**: compare every A-descriptor to every B-descriptor. Exact, $O(N \cdot M)$.
- **kNN match**: return the *two* nearest neighbours per descriptor — needed for the ratio test.
- **FLANN**: approximate nearest neighbour (kd-trees / LSH). Much faster for thousands of features.

Key idea

Small sets: brute-force is fine and exact. Large sets / real-time: FLANN trades a little accuracy for big speed.

Rejecting bad matches

Rejecting bad matches

- Many nearest-neighbour matches are wrong. Filter them:
- **Ratio test** (Lowe): keep a match only if the best is much closer than the 2nd-best.
- **Cross-check**: keep only mutual nearest neighbours.
- **RANSAC** (Week 11): reject geometric outliers by fitting a transform.

✕ Pitfall

Raw matches are noisy. Without the ratio test / RANSAC, a few wrong matches wreck the alignment.

Lowe's ratio test — worked example

For a descriptor in A, its two nearest neighbours in B have distances d_1 (best), d_2 (2nd):

$$\text{keep if } \frac{d_1}{d_2} < 0.75.$$

✔ Distinctive — keep

$d_1 = 30,\ d_2 = 90$: ratio $= 0.33 < 0.75$. The best is clearly better ⇒ trust it.

✖ Ambiguous — reject

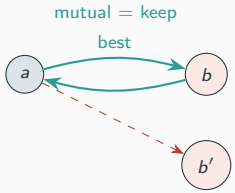
$d_1 = 80,\ d_2 = 88$: ratio $= 0.91 > 0.75$. Two candidates almost tie ⇒ unreliable, drop it.

Key idea

The ratio test asks "is this match *uniquely* good?" — it kills matches in repetitive texture.

Cross-check — mutual nearest neighbours

- A's best match in B is b; B's best match for b back in A must be the same a.
- Only *mutual* best matches survive — a cheap, symmetric filter.
- OpenCV: `BFMatcher(..., crossCheck=True)` (do not combine with the ratio test).



RANSAC — a one-slide preview (Week 11)

- Even after ratio/cross-check, some geometric outliers remain.
- **RANSAC**: repeatedly pick a minimal random sample, fit a transform, count **inliers** (matches that agree with it), keep the best-supported model.
- The consensus transform (a homography) both filters outliers *and* aligns the images.

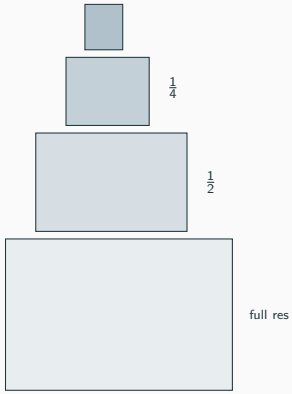
Key idea

Ratio test / cross-check clean up *descriptor* errors; RANSAC cleans up *geometry* errors. You need both.

Scale invariance & applications

Scale invariance: the pyramid

- To find features at any size, build an **image pyramid** (progressively smaller copies).
- Detect at each level; the scale where a feature is strongest is its "characteristic scale."
- That is how SIFT/ORB match a small template to a large photo.



How many matches do you need?

- To estimate a **homography** (Week 11) you need ≥ 4 good correspondences.
- In practice, gather dozens of matches, then let **RANSAC** pick a consistent set of 4+.
- More good matches $\rightarrow$ a more stable, accurate transform.

Key idea

Quantity gives robustness: many matches let outlier rejection succeed even when a third are wrong.

Applications of features

- **Image stitching** (panoramas): match features, find the transform, blend.
- **Object recognition**: match a template's features to a scene.
- **Tracking, SLAM, AR**: follow features across video to estimate motion.
- **DocScan**: match the page/template to align and rectify (Week 11).

Classical features vs deep features

- Classical (SIFT/ORB): hand-designed, interpretable, no training — still excellent for geometry.
- Deep (learned descriptors, SuperPoint): better under extreme appearance change, need training.
- For alignment/geometry, classical features remain a strong, practical choice.

Key idea

Features are how a computer says "this point here is the same as that point there" — the foundation of geometric vision.

In-class exercise

In-class exercise (10 min)

Reason about matching

1. Two ORB descriptors: 1100 1010 and 1110 0011. Hamming distance?
2. A match has $d_1 = 60$, $d_2 = 66$. Does Lowe's ratio test (0.75) keep it? Why?
3. You match a template plate to a photo taken at an angle and half the size. Which descriptor properties make this possible?

Debrief

- (1) XOR = 0010 1001, popcount = 3 $\Rightarrow$ Hamming distance **3**.
- (2) Ratio = $60/66 = 0.91 > 0.75 \Rightarrow$ **reject** — the two candidates nearly tie, so it is ambiguous.
- (3) **Scale and rotation invariance** — the descriptor is the same despite size/angle change.

Key idea

Good features + good matches + outlier rejection = reliable correspondence, the basis of alignment.

Wrap-up & deliverable

Features in code

```python
orb = cv2.ORB_create(500)
kp1, des1 = orb.detectAndCompute(img1, None)
kp2, des2 = orb.detectAndCompute(img2, None)
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = sorted(bf.match(des1, des2), key=lambda m: m.distance)
# next week: cv2.findHomography(pts1, pts2, cv2.RANSAC) to align
```

Detect $\rightarrow$ describe $\rightarrow$ match $\rightarrow$ (Week 11) estimate the transform.

Matching with the ratio test

```python
bf = cv2.BFMatcher(cv2.NORM_HAMMING)          # no crossCheck for kNN
knn = bf.knnMatch(des1, des2, k=2)            # 2 nearest per descriptor
good = [m for m, n in knn if m.distance < 0.75 * n.distance]
print(len(good), "good matches after Lowe's ratio test")
```

✔ Good practice

Use `knnMatch` + ratio test *or* `crossCheck=True` — not both at once.

What you should now be able to do

- Explain how SIFT and ORB build their descriptor vectors.
- Match descriptors with the right metric (Euclidean vs Hamming), brute-force or FLANN.
- Reject bad matches with the ratio test and cross-check; explain RANSAC's role.
- Connect scale/rotation invariance to real matching across views.

This week's deliverable

Due before Week 11 (see `project/`)

1. Detect Harris corners and ORB keypoints on real images; visualize.
2. Match ORB features between an image and a rotated/scaled copy; count good matches.
3. Apply the ratio test / cross-check; show how it removes wrong matches.
4. Note: how will feature matching help align a photographed page in DocScan?

Features are “the same point, seen again.”

Detect distinctive points, describe them invariantly, match across images, reject outliers.